

LAPORAN PRAKTIKUM

STRATEGI ALGORITMA

PRAKTIKUM 5: ALGORITMA DECREASE & CONQUER

I. DEFINISI

1.1 Algoritma Decrease and Conquer

Definisi: Decrease and Conquer adalah metode perancangan algoritma yang mereduksi persoalan berukuran n menjadi beberapa subpersoalan yang lebih kecil, tetapi selanjutnya hanya memproses satu subpersoalan saja secara rekursif. Berbeda dengan Divide and Conquer yang memproses semua subpersoalan dan menggabungkan seluruh solusinya, Decrease and Conquer tidak memiliki tahap combine.

Dua Tahap:

1. **Decrease:** mereduksi ukuran masalah menjadi subpersoalan yang lebih kecil, biasanya dengan mengurangi satu elemen atau membagi setengah.
2. **Conquer:** memproses satu subpersoalan secara rekursif hingga mencapai basis penyelesaian langsung.

Tiga Varian:

3. **Decrease by a constant:** ukuran masalah berkurang sebesar konstanta tetap setiap iterasi (biasanya 1). Contoh: Selection Sort, Insertion Sort, pencarian sekuensial.
4. **Decrease by a constant factor:** ukuran masalah berkurang sebesar faktor tetap setiap iterasi (biasanya 2). Contoh: Binary Search, Fake Coin Problem.
5. **Decrease by a variable size:** ukuran pengurangan bervariasi tiap iterasi. Contoh: BST search, Selection Problem.

Fungsi: Digunakan untuk menyelesaikan persoalan yang dapat disederhanakan secara bertahap dengan mereduksi ukuran masalah, seperti pengurutan, pencarian, dan traversal.

Kelebihan: Lebih sederhana dari D&C karena tidak memerlukan tahap combine, overhead rekursi lebih kecil, dan cocok untuk masalah yang solusinya dapat ditemukan dengan mereduksi satu elemen setiap langkah.

Kekurangan: Tidak seefisien D&C untuk masalah yang secara alami dapat diparalelkan, dan kompleksitas bisa tetap $O(n^2)$ apabila strategi pereduksian tidak optimal.

1.2 Selection Sort sebagai Decrease and Conquer

Selection Sort adalah contoh klasik varian Decrease by a Constant ($k=1$). Pada setiap langkah, masalah pengurutan array $A[i..j]$ direduksi menjadi submasalah $A[i+1..j]$ dengan cara meletakkan elemen terkecil di posisi i melalui fungsi Bagi. Tidak ada tahap combine, karena posisi $A[i]$ sudah final setelah fungsi Bagi dijalankan.

Kompleksitas waktu: $T(n) = T(n-1) + (n-1)$ sehingga $T(n) = n(n-1)/2 = O(n^2)$.

II. PRE TEST

Pertanyaan 1: Apakah karakteristik dari algoritma Decrease and Conquer?

Jawaban: Karakteristik utama Decrease and Conquer adalah: (1) masalah berukuran n direduksi menjadi satu submasalah yang lebih kecil, (2) hanya satu submasalah yang diproses secara rekursif, (3) tidak ada tahap combine karena solusi submasalah langsung menjadi bagian dari solusi masalah asli, dan (4) proses pereduksian berlangsung secara bertahap hingga mencapai basis kasus yang dapat diselesaikan langsung.

Pertanyaan 2: Jelaskan perbedaan algoritma Decrease and Conquer dengan Divide and Conquer!

Aspek	Decrease and Conquer	Divide and Conquer
Pembagian submasalah	Menjadi submasalah lebih kecil	Menjadi dua atau lebih submasalah
Jumlah submasalah diproses	Hanya satu submasalah	Semua submasalah diproses
Tahap Combine	Tidak ada	Ada, hasil digabungkan
Ukuran pereduksian	Constant, factor, atau variable	Biasanya $n/2$ atau n/k
Contoh algoritma	Selection Sort, Binary Search	Merge Sort, Quick Sort, MinMaks

Pertanyaan 3: Sebutkan algoritma selain Selection Sort yang menggunakan teknik Decrease and Conquer!

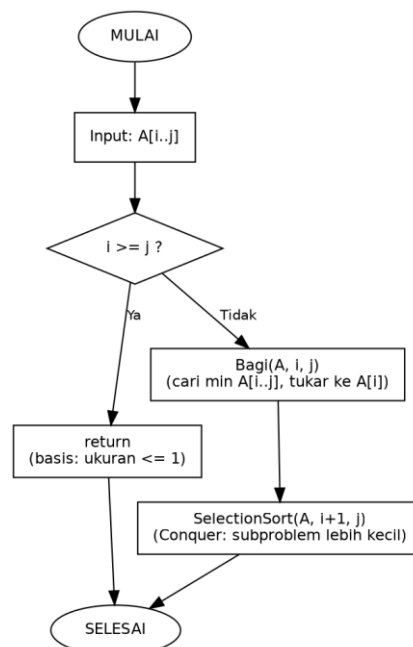
Jawaban:

- **Insertion Sort** (decrease by constant 1): setiap langkah menyisipkan satu elemen ke subarray yang sudah terurut.
- **Binary Search** (decrease by constant factor 2): setiap langkah mengeliminasi setengah ruang pencarian berdasarkan perbandingan dengan elemen tengah.
- **Topological Sort** (decrease by constant 1): setiap langkah memilih satu simpul tanpa dependensi masuk dan menghapusnya dari graf.
- **BST Search** (decrease by variable size): setiap langkah memilih subtree kiri atau kanan berdasarkan perbandingan nilai.
- **Euclid GCD** (decrease by variable size): setiap langkah mengganti pasangan (m, n) dengan $(n, m \bmod n)$ hingga remainder = 0.

III. ANALISIS PRAKTIKUM

3.1 Langkah Praktikum: Selection Sort D&C

Flowchart



Gambar 3.1 Flowchart Algoritma Selection Sort Decrease and Conquer

Source Code Python

```
def bagi(arr, i, j):  
    # cari indeks elemen terkecil dari A[i..j]  
    idx_min = i  
    for k in range(i + 1, j + 1):  
        if arr[k] < arr[idx_min]:
```

```

        idx_min = k

        # tukar elemen terkecil ke posisi i
        arr[i], arr[idx_min] = arr[idx_min], arr[i]

def selection_sort_dc(arr, i, j):
    # basis: ukuran subarray <= 1, sudah terurut
    if i >= j:
        return

    # decrease: letakkan elemen terkecil A[i..j] ke posisi i
    bagi(arr, i, j)

    # conquer: urutkan sisa subarray A[i+1..j] secara rekursif
    selection_sort_dc(arr, i + 1, j)

if __name__ == "__main__":
    arr = [64, 25, 12, 22, 11]
    print("Data awal      :", arr)

    selection_sort_dc(arr, 0, len(arr) - 1)

    print("Data terurut:", arr)

```

Analisis Output

Program dijalankan dengan data awal [64, 25, 12, 22, 11] sesuai modul:

Rekursi ke	Subarray A[i..j]	Elemen Min	Setelah Bagi	Subproblem berikutnya
1	A[0..4] = [64,25,12,22,11]	11 (idx 4)	[11,25,12,22,64]	A[1..4]
2	A[1..4] = [25,12,22,64]	12 (idx 2)	[11,12,25,22,64]	A[2..4]
3	A[2..4] = [25,22,64]	22 (idx 3)	[11,12,22,25,64]	A[3..4]
4	A[3..4] = [25,64]	25 (idx 3)	[11,12,22,25,64]	A[4..4]
5	A[4..4] = [64]	basis i>=j	tidak ada tukar	selesai

Output akhir: [11, 12, 22, 25, 64]. Setiap rekursi mereduksi submasalah sebesar 1 elemen (decrease by constant = 1). Tidak ada tahap combine karena posisi A[i] sudah final setelah setiap pemanggilan Bagi.

3.2 Post Test: Selection Sort dengan Input Pengguna

Source Code Python

```

def bagi(arr, i, j):
    # cari indeks elemen terkecil dari A[i..j]
    idx_min = i
    for k in range(i + 1, j + 1):
        if arr[k] < arr[idx_min]:
            idx_min = k

    # tukar elemen terkecil ke posisi i
    arr[i], arr[idx_min] = arr[idx_min], arr[i]

```

```

def selection_sort_dc(arr, i, j):
    # basis: ukuran subarray <= 1, sudah terurut
    if i >= j:
        return

    # decrease: letakkan elemen terkecil ke posisi i
    bagi(arr, i, j)

    # conquer: rekursif pada subarray yang lebih kecil
    selection_sort_dc(arr, i + 1, j)

    # terima input array dari pengguna
    raw = input("Masukkan bilangan dipisah spasi: ")
    arr = list(map(int, raw.split()))

    print("Data awal      :", arr)
    selection_sort_dc(arr, 0, len(arr) - 1)
    print("Data terurut:", arr)

```

Analisis Output:

Program menerima input dari pengguna berupa bilangan yang dipisah spasi, mengonversinya ke list integer, lalu menjalankan Selection Sort D&C. Contoh eksekusi dengan input "5 3 8 1 9 2":

Input	Data Awal	Data Terurut
5 3 8 1 9 2	[5, 3, 8, 1, 9, 2]	[1, 2, 3, 5, 8, 9]

Implementasi ini membuktikan bahwa Selection Sort berbasis Decrease and Conquer menghasilkan output yang identik dengan versi iteratif, dengan alur rekursi yang lebih eksplisit mencerminkan prinsip decrease by a constant.

IV. IMPLEMENTASI NYATA

Pendekatan Decrease and Conquer pada Selection Sort telah dikaji secara akademis untuk penerapan paralel pada sistem komputasi modern. Jurnal berikut mendukung relevansinya.

[1] **Penulis:** Ganapathi, P. dan Chowdhury, R. (2022)

Judul: *"Parallel Divide-and-Conquer Algorithms for Bubble Sort, Selection Sort and Insertion Sort"*

Publikasi: The Computer Journal, Vol. 65, No. 10, Hal. 2709-2719. Oxford University Press. DOI: 10.1093/comjnl/bxab107

Intisari: Makalah ini menyajikan algoritma rekursif berbasis Decrease and Conquer untuk Selection Sort, Bubble Sort, dan Insertion Sort yang diparalelkan menggunakan Intel

Cilk Plus dengan 32 thread. Hasil eksperimen menunjukkan Selection Sort rekursif D&C mencapai speedup hingga ratusan kali dibanding versi sekuensialnya pada input terurut terbalik, membuktikan bahwa struktur D&C secara inheren mendukung paralelisme meskipun kompleksitas asimtotiknya tetap $O(n^2)$.

Link: <https://academic.oup.com/comjnl/article/65/10/2709/6334046>

Kelebihan dalam konteks nyata:

- Struktur rekursif Decrease and Conquer secara alami mendukung paralelisasi, memungkinkan speedup signifikan pada sistem multi-core untuk data berukuran sedang.
- Implementasi rekursif lebih mudah dibuktikan kebenarannya secara formal dibanding versi iteratif, penting dalam sistem kritis seperti avionik atau medis.
- Cocok untuk dataset kecil hingga menengah pada embedded system yang memerlukan algoritma sederhana dengan footprint memori minimal.

Kekurangan dalam konteks nyata:

- Kompleksitas $O(n^2)$ tetap tidak berubah meskipun direkursifkan, sehingga tidak disarankan untuk dataset besar dibanding Merge Sort atau Quick Sort.
- Overhead pemanggilan fungsi rekursif membutuhkan stack memori tambahan, yang perlu diperhatikan pada sistem dengan memori terbatas.